TRƯỜNG ĐẠI HỌC BÁCH KHOA
ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần 13 - Học phần: Lập trình Đa nền tảng

Chủ đề 11: Chia sẻ trạng thái giữa các widget

Giảng viên hướng dẫn: TS. Nguyễn Duy Nhật Viễn

Sinh viên thực hiện: Lê Anh Khoa (MSSV: 106230016)
Ngô Quang Thế (MSSV: 106230028)

Lớp sinh hoạt: 23KTMT1

Đà Nẵng, 2026

Nội dung báo cáo

1. **Phân loại trạng thái và nguyên tắc sở hữu**
   Phân biệt Ephemeral vs App State; Single Source of Truth; quy tắc đặt state tại LCA.
2. **Truyền dữ liệu và vấn đề Prop Drilling**
   Cơ chế Constructor/Callback; phân tích code nhận truyền tiếp và hiện tượng build lan truyền.
3. **Các cơ chế chia sẻ trạng thái trong Flutter**
   Cấu trúc InheritedWidget tra cứu trực tiếp; đặc tính của ValueNotifier và ChangeNotifier.
4. **Thực nghiệm: Ứng dụng TaskFlow & So sánh thực tế**
   Đo lường số lần build giữa 2 trường hợp, so sánh và chọn cách dùng phù hợp.

Phân công công việc

STT	Họ và tên	Nội dung đảm nhiệm	Slide	Đóng góp
01	Ngô Quang Thế 106230028	Phân loại trạng thái, Nguyên tắc LCA, Vấn đề Prop Drilling, Cơ chế InheritedWidget, ChangeNotifier, ValueNotifier	1–11	50%
02	Lê Anh Khoa 106230016	Ứng dụng TaskFlow, Thực nghiệm Trường hợp 1 & 2, Đo lường số lần build, Cải tiến ValueNotifier & Kết luận	12–21	50%

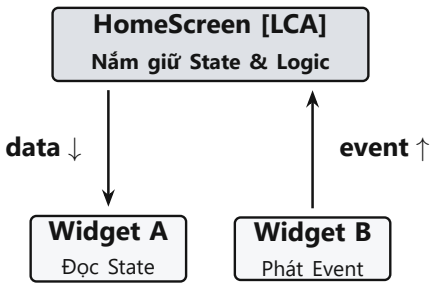
Hai loại trạng thái: Ephemeral và App State

Ephemeral State (Cục bộ)
Tạm thời trong 1 widget
(Tab, Expansion, Form)

App State (Chia sẻ)
Dùng chung nhiều nơi (Danh sách việc, Tiến độ, Phân bổ)

- **Ephemeral State**: Dữ liệu tạm thời, gói gọn trong một widget (dùng `setState()`).
- **App State**: Dữ liệu dùng chung cần chia sẻ giữa nhiều màn hình hoặc widget khác nhánh.

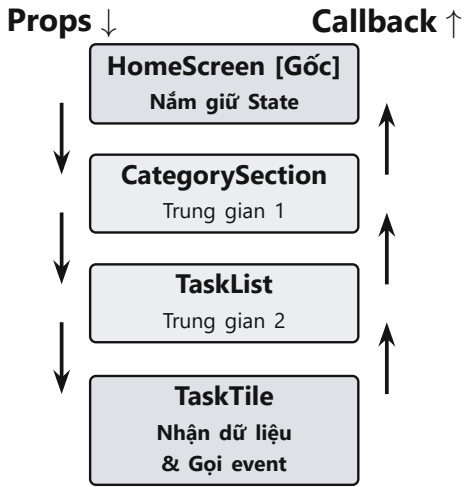
Nguyên tắc sở hữu: Nơi đặt State và Logic



Hai nguyên tắc xác định nơi đặt State:

- **Single Source of Truth**: Luôn giữ dữ liệu tại một nơi duy nhất để tránh bị lệch trạng thái giữa các màn hình.
- **Đặt ở cha chung gần nhất (LCA)**: Nâng state lên mức thấp nhất mà các widget con cần dùng vẫn truy cập được.

Truyền dữ liệu và Vấn đề Prop Drilling

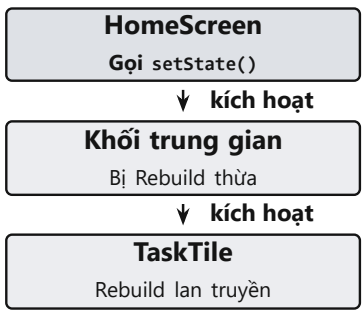



```
// Widget trung gian KHÔNG dùng dữ liệu,
// chỉ nhận để truyền tiếp xuống con:
const CategorySection({
  required this.tasks,
  required this.onToggle,
});

TaskList(tasks: tasks, onToggle: onToggle);
```

Truyền qua Constructor & Callback:
Widget trung gian nhận qua constructor và callback chỉ để truyền tiếp, gây rườm rà (Prop Drilling).

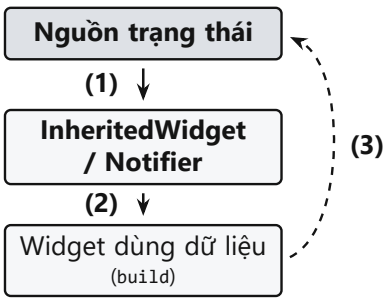
Tác động hiệu năng: Rebuild lan truyền




```
void _toggleTask(String id) {
  // Gọi setState() tại gốc cây:
  setState(() {
    _tasks = _tasks.map((t) => t.id == id
        ? t.copyWith(isCompleted: !t.isCompleted)
        : t).toList();
  });
}
```

Hậu quả: Gọi `setState()` ở gốc khiến các widget trung gian và toàn bộ cây con phải build lại thừa.

Chu trình chia sẻ trạng thái trong Flutter



(1) Nguồn dữ liệu duy nhất
Nắm giữ dữ liệu gốc và xử lý logic

(2) Tra cứu trực tiếp
Widget con lấy dữ liệu từ widget cấp trên hoặc qua Notifier

(3) Chỉ build lại widget cần dùng
Không kéo theo các widget trung gian build lại

InheritedWidget: Cấu trúc và Tra cứu $O(1)$

```
class TaskScope extends InheritedWidget {
  final List<TaskItem> tasks;

  static TaskScope of(BuildContext ctx) =>
    ctx.dependOnInheritedWidgetOfExactType<TaskScope>()!;

  @override
  bool updateShouldNotify(TaskScope old) =>
    tasks != old.tasks;
}
```

1. Tra cứu trực tiếp $O(1)$

Widget con tra cứu trực tiếp qua bảng băm mà không cần duyệt ngược cây.

2. updateShouldNotify

Chỉ kích hoạt rebuild cho widget phụ thuộc khi dữ liệu thực sự thay đổi.

Phân biệt: Đăng ký lắng nghe vs Chỉ đọc dữ liệu

CÓ Rebuild

Phương thức:
`dependOnInheritedWidgetOfExactType<T>()`

Hành vi:
Lấy dữ liệu và **đăng ký quan hệ phụ thuộc**.

Tình huống sử dụng:
Thẻ hiển thị (`TaskTile`, Bảng tiến độ): tự động build lại khi dữ liệu cập nhật.

KHÔNG Rebuild

Phương thức:
`getInheritedWidgetOfExactType<T>()`

Hành vi:
Chỉ lấy tham chiếu (instance) để gọi hàm, **KHÔNG đăng ký**.

Tình huống sử dụng:
Nút bấm hoặc sự kiện (`onToggle`): gọi hàm cập nhật, bản thân nút không cần build lại.

ChangeNotifier và ValueNotifier

ValueNotifier<T>

- **Đối tượng**: Một biến đơn (`bool`, `int`).
- **Cơ chế**: Tự phát khi gán lại `value`.
- **Widget**: `ValueListenableBuilder`.

Phù hợp cho trạng thái đơn giản như công tắc, thanh trượt hay biến đếm.

ChangeNotifier

- **Đối tượng**: Model gồm nhiều biến.
- **Cơ chế**: Gọi hàm `notifyListeners()`.
- **Widget**: `ListenableBuilder`.

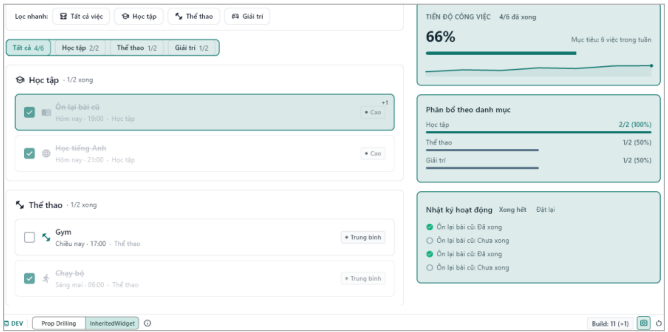
Phù hợp cho dữ liệu phức tạp gồm nhiều thuộc tính và hành vi.

So sánh 3 cơ chế chia sẻ trạng thái

Tiêu chí	InheritedWidget	ValueNotifier	ChangeNotifier
Bản chất	Phân phối dữ liệu trong cây	Báo khi một biến đổi	Báo khi Model nhiều biến đổi
Tra cứu	Tra cứu qua `BuildContext`	Độc lập với cây widget	Độc lập với cây widget
Nơi giữ state	Widget cha giữ	Tự giữ biến `value`	Tự giữ các biến và logic
Cập nhật UI	Widget con tự đăng ký	ValueListenableBuilder	ListenableBuilder

Quan hệ: `ValueNotifier<T>` kế thừa từ `ChangeNotifier`, được tối ưu riêng cho trường hợp quản lý một biến.

Ứng dụng TaskFlow



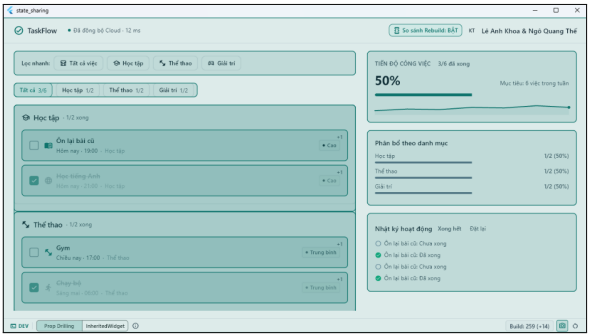
Nhìn vào đâu: Bấm hoàn thành việc ở **Danh sách việc** (trái) → **Tiến độ** (phải) tự động cập nhật.

Trường hợp 1: Kỹ thuật Prop Drilling

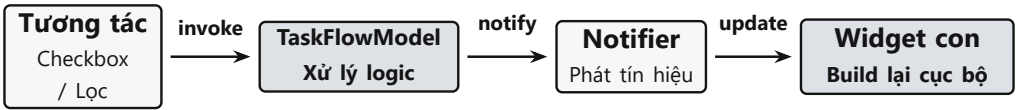
```
// Gọi setState() ở gốc HomeScreen:
void _handleToggleTask(String id) {
  setState(() {
    _model.toggleTask(id);
  }); // Toàn bộ cây con build lại!
}
```

Build lan truyền: +14

Toàn bộ màn hình, nhóm danh mục và 6 thẻ đều bị build lại thừa.



Luồng trạng thái trong ứng dụng TaskFlow

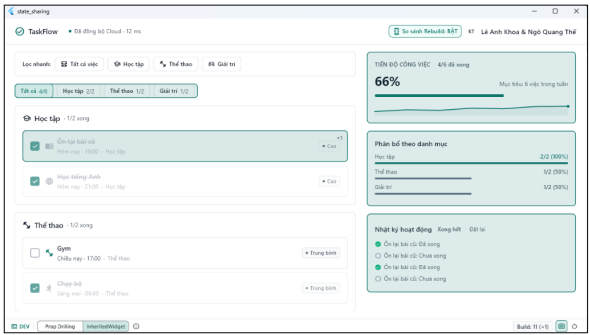


- **Khi người dùng tương tác**: Checkbox hoặc bộ lọc gọi hàm trên `TaskFlowModel` để cập nhật dữ liệu.
- **Khi dữ liệu đổi**: Model gọi `notifyListeners()`, chỉ những widget đang đăng ký nghe mới build lại.

```
void toggleTask(String id) {
  _tasks[i] = _tasks[i].copyWith(...);
  notifyListeners(); // Báo cho UI
}
```

Trường hợp 2: InheritedWidget kết hợp Model

```
// Thẻ lá tự đọc dữ liệu từ Scope O(1):
Widget build(BuildContext context) {
  final scope = TaskScope.of(context);
  final task = scope.tasks
      .firstWhere((t) => t.id == widget.id);
  return _buildTile(task);
}
```



Build cục bộ: +1

Chỉ duy nhất thẻ được click build lại, các widget khác không bị ảnh hưởng.

Kết quả đo lường và đánh giá thực nghiệm

14 → 1
Lượt build khi tick việc

-92,9%
Giảm build thừa (khi tick 1 việc)

2 → 0
Widget trung gian

Thao tác thử nghiệm	Trường hợp 1 (Prop Drilling)	Trường hợp 2 (InheritedWidget)	Hiệu quả cải thiện
Tick 1 công việc	14	1	**Giảm 92,9%** build thừa
Lọc nhanh danh mục	14	4	**Giảm 71,4%** build thừa
Xong tất cả / Đặt lại	14	8	**Giảm 42,9%** build thừa

Kết luận: Trường hợp **InheritedWidget kết hợp Model** giúp giảm 42,9–92,9% lượt build thừa so với Trường hợp 1.

Cập nhật cục bộ từng thẻ với ValueNotifier

```
// Tách Notifier riêng cho từng ID:
void toggleTask(String id) {
  _taskNotifiers[id]?.value = updated;
}

// Bọc Builder tại cấp lá:
ValueListenableBuilder<TaskItem>(
  valueListenable: model.getNotifier(id),
  builder: (context, item, _) => TaskTile(item),
);
```

Hướng cải tiến: Tách Notifier cho từng thẻ

- **Cơ chế**: Gán cho mỗi thẻ công việc một `ValueNotifier` riêng biệt thay vì dùng chung cả Model.
- **Hiệu quả**: Khi tick một việc, chỉ build lại đúng thẻ có thay đổi thông qua `ValueListenableBuilder`.

Kết luận: Khi nào dùng cơ chế nào

Cơ chế	Dùng khi	Đặc điểm
Constructor / Callback	Cha–con 1–2 tầng	Đơn giản, không thêm class phụ
InheritedWidget	Toàn cây, state dùng chung	Tra cứu trực tiếp $O(1)$, Theme, Config
Change / ValueNotifier	Dữ liệu hay thay đổi, có logic	Chỉ build lại widget lắng nghe

Nên nhớ: Bắt đầu từ cách đơn giản nhất, chỉ dùng Notifier hoặc InheritedWidget khi cần truyền qua nhiều tầng.

Tài liệu tham khảo

1. Flutter Documentation, *"Differentiate between ephemeral state and app state"*, Google. `https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app`
2. Flutter Documentation, *"InheritedWidget class"*, Google. `https://api.flutter.dev/flutter/widgets/InheritedWidget-class.html`
3. Flutter Documentation, *"ListenableBuilder class"*, Google. `https://api.flutter.dev/flutter/widgets/ListenableBuilder-class.html`
4. Flutter Documentation, *"ValueNotifier<T> class"*, Google. `https://api.flutter.dev/flutter/foundation/ValueNotifier-class.html`
5. TS. Nguyễn Duy Nhật Viễn, *"Bài giảng Lập trình Đa nền tảng"*, Khoa Điện tử và Trí tuệ Nhân tạo, Trường Đại học Bách khoa — Đại học Đà Nẵng, 2026.

> **Mã nguồn chương trình thực nghiệm trên GitHub**
>
> `https://github.com/KhoaLe246205/cross-platform/tree/main/state_sharing`

Cảm ơn Thầy và các bạn đã lắng nghe!